

Datenbanken

Prof. Dr. Philipp Bruland

Technische Hochschule Ostwestfalen-Lippe
Fachbereich: Informatik und Automation

Sommersemester 2026

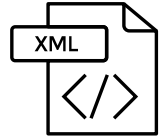
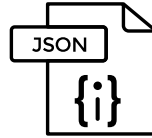
Lernziele

- Was sind überhaupt Daten und wie können Sie digital gespeichert werden?
- Kennenlernen von XML / JSON
- Wie lassen sich die Metadaten in XML beschreiben?
- Welche Möglichkeiten des Zugriffs auf XML-Dokumente gibt es?

Übersicht

- 1 XML-Technologien
 - Einführung in XML
 - XML-Schema (DTD & XSD)
 - XPath - Zugriff auf XML-Dokumente

Unterschiedliche Datenformate



Motivation für XML / JSON



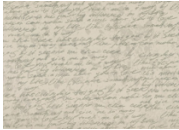
Übertragungs- und Spezifizierungsformat für unterschiedlichste Bereiche:

- Produktlebenszyklen
- Klinische Studien-Daten
- Medizinische Behandlungsdaten
- Architektur-Daten
- Elektronische Prozesse
- Bücher
- Grafiken (SVG)
- Nachrichten

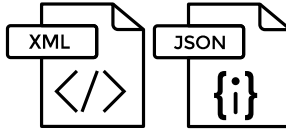


Einleitung in semi-strukturierte Daten (XML, JSON)

Unstrukturiert (Prosa Text)



Semi-strukturiert



(stark) strukturiert
(Datenbanken)



- Semi-strukturierte Daten
 - Kompromiss zwischen unstrukturierten und stark strukturierten Daten
 - Hohe Flexibilität
 - Text- und Daten-orientierte Dokumente möglich
 - Optionales Schema
- Regelmäßige Struktur durch Schema
 - Datensätze mit gleichartigem Aufbau
 - Leichtere und effiziente automatische Verarbeitung von Daten
 - Durch Schema-Konventionen ist der Inhalt für „jede:n“ nachvollziehbar
 - Weniger flexibel

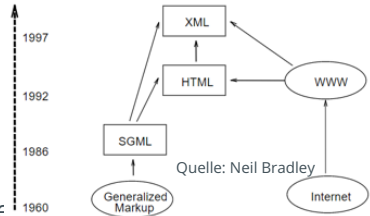
Was aber bedeutet „semi-strukturierte Daten“?

- Die Struktur der Daten ist unregelmäßig.
- Das Schema ist implizit in den Daten enthalten.
- Die Struktur der Daten ist unvollständig.
- Das Schema ist flexibel.
- Das Schema kann relativ groß werden.
- Das Schema unterliegt iterativen Änderungen.

Quelle: Abiteboul, 1997

Repräsentation von semistrukturierten Daten / Dokumenten

- Markup-Sprache: Dokumentabschnitte werden durch Marker ausgezeichnet
 - SGML (Standard Generalized Markup Language): hohe Komplexität
 - HTML (HyperText Markup Language): SGML-Anwendung, feste Menge an Auszeichnungselementen
- **XML (eXtensible Markup Language)**
 - Empfehlung des W3C - World Wide Web Consortium
 - SGML-Kompatibilität
 - Klare und lesbare Dokumentformate
 - Schemaunterstützung (DTD, XML Schema(XSD))
 - Abfragemöglichkeiten (XPath, XQuery)
 - XML-Datenbanken / SQL-Integration
- Starke XML-Verbreitung u. a. zum Datenaustausch
- **JSON (JavaScript Object Notation)** (Leichtgewichtige)
 - Schemalose Repräsentation von Dokumenten / geschachtelten Datenobjekten
 - Unterstützung in SQL und durch Document Stores (NoSQL-Datenbanken), z. B.: MongoDB



XML-Dokumente

- Eigentlich zur Kommunikation zwischen Menschen eingesetzt
 - Mensch ↔ Mensch
 - Natürliche (menschenslesbare) Sprache wird verwendet
 - Beinhaltet aber auch komplexe und irreguläre Strukturen
- Elektronische Verarbeitung: auch für die Kommunikation:
 - Computer ↔ Computer
 - Datenzentriert
 - Mensch ↔ Computer
 - Dokumentenzentriert
 - XML kann zur Darstellung und zum Transport dieser Informationen eingesetzt werden.

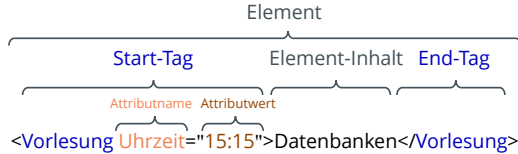
XML - Beispiel

- XML-Dokumente besitzen eine Baumstruktur
 - Schachtelung von Elementen (= Baumknoten)
 - Elemente können neben Unterelementen Attribute bzw. Inhalt (z. B. Text) aufweisen

XML Beispieldokument

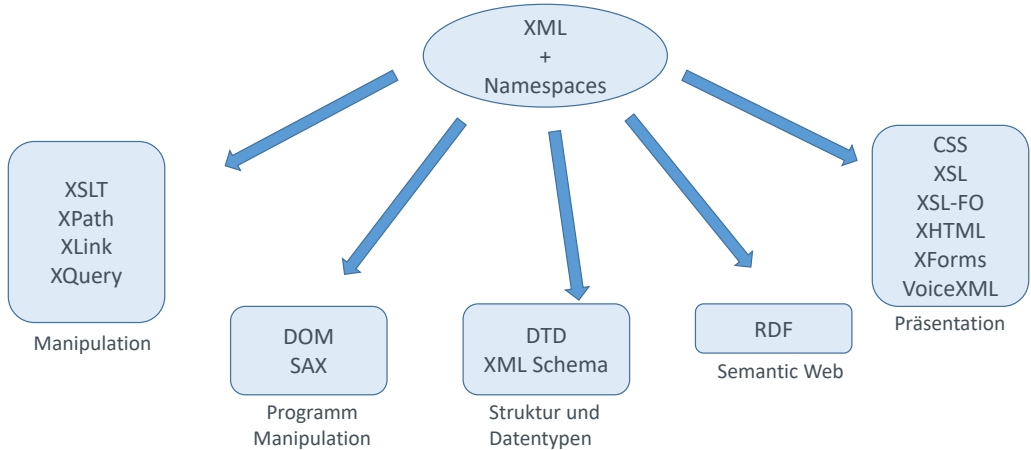
```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE Vorlesungsverzeichnis SYSTEM "http://db.th-owl.de/dtd/VLVerzeichnis.dtd">
3 <VLVerzeichnis>
4   <Vorlesung Uhrzeit="11:35">
5     <Thema>DB</Thema>
6     <Dozent>
7       <Name>Prof. Bruland</Name>
8       <Einrichtung>TH OWL</Einrichtung>
9     </Dozent>
10  </Vorlesung>
11 </VLVerzeichnis>
```

XML: Elemente und Attribute



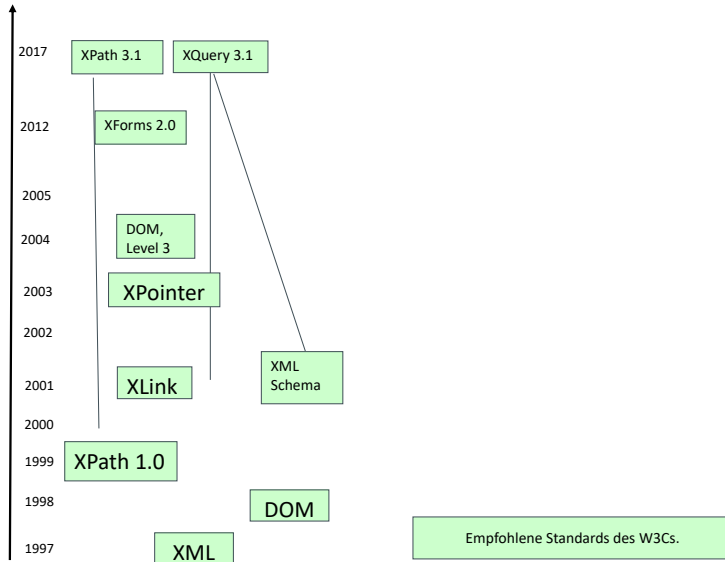
- Elemente
 - Start- und End-Tag müssen vorhanden sein.
 - Ausnahme: leeres Element (Bsp. `<Leer />`)
 - feste Reihenfolge von Geschwisterelementen (Reihenfolge ist wichtig!)
 - Element-Inhalt besteht entweder nur aus weiteren Elementen (**element content**) oder aus Zeichendaten optional vermischt mit Elementen (**mixed content**)
- Attribute
 - Attributwerte können nicht strukturiert werden
 - Attributreihenfolge beliebig
- Groß-/Kleinschreibung ist relevant (gilt für alle Bezeichner in XML)
- weitere XML-Bestandteile (hier nicht behandelt): Entities, Processing Instructions, Kommentare

Wer/Was befindet sich noch in der X-Familie



Frank P. Coyle. XML, Web Services, and the Data Revolution.

Zeitliche Entwicklung von XML



Übersicht

1 XML-Technologien

Einführung in XML

XML-Schema (DTD & XSD)

XPath - Zugriff auf XML-Dokumente

Doch wo kommt nun die (semi) Struktur her?

- Bei XML gibt es zwei Varianten, die Struktur in sogenannten Meta-Daten anzugeben:
- DTD (Document Type Definition)
 - Älteres Format, welches sich immer noch finden lässt
- XSD (XML Schema Definition)
 - Wird hauptsächlich nur noch verwendet, da es mehr Möglichkeiten bietet.

XML-Strukturdefinition: DTD

- DTD (Document Type Definition) / Schema optional
- XML-Dokumente sind wohlgeformt, d. h. syntaktisch korrekt
 - alle (außer leere) Elemente müssen ein Start-Tag und ein Ende-Tag haben
 - korrekte Schachtelung von Tags
 - eindeutige Attributnamen pro Element; Attributwerte in Hochkommas ...
- XML-Daten sind gültig: wohlgeformt und DTD- bzw. Schema-konform

XML Beispieldokument

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE Vorlesungsverzeichnis SYSTEM "
   http://db.th-owl.de/dtd/VLVerzeichnis
   .dtd">
3 <VLVerzeichnis >
4   <Vorlesung Uhrzeit="14:15">
5     <Thema>DB</Thema>
6     <Dozent>
7       <Name>Prof. Bruland</Name>
8       <Einrichtung>TH OWL</Einrichtung>
9     </Dozent>
10   </Vorlesung>
11 </VLVerzeichnis>
```

VLVerzeichnis.dtd:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!ELEMENT VLVerzeichnis (Vorlesung)*>
3 <!ELEMENT Vorlesung (Thema, Dozent)>
4 <!ATTLIST Vorlesung Uhrzeit CDATA #
   REQUIRED>
5 <!ELEMENT Thema (#PCDATA)>
6 <!ELEMENT Dozent (Name, Einrichtung?)>
7 <!ELEMENT Name (#PCDATA)>
8 <!ELEMENT Einrichtung (#PCDATA)>
```

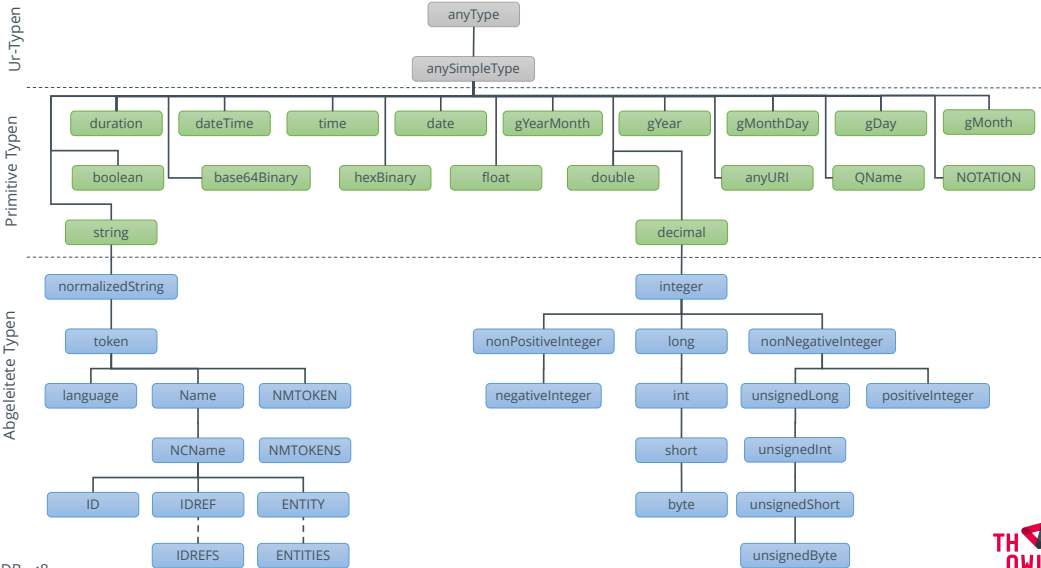

XML Schema-Sprachen

- Schemainformationen wichtig für ...
 - Korrektheit von Daten/Dokumenten: Validierung von Struktur und Datentypen
 - Effiziente DB-Speicherung und Verarbeitung von Anfragen
 - Interoperabilität in verteilten Umgebungen: feste Formatierungen für Datenaustausch (z. B. in Schnittstellen für Web-Services)
- Defizite der DTD:
 - Eigenständige Syntax (kein XML-Format) → extra Parser notwendig
 - Kaum Typisierungen von Elementinhalt (Text) und Attributwerten möglich
 - Mehrere DTDs pro Dokument sind nicht erlaubt
 - Keine Unterstützung von Namensräumen
 - Unzureichende Unterstützung der Interoperabilität und Wiederverwendbarkeit

XML Schema Definition (XSD)

- XML Schemabeschreibungssprache, entwickelt durch das W3C
 - Recommendation seit 2001 , 2012: V1.1
 - XML Schema Definition (XSD)
- Eigenschaften
 - Verwendet XML-Syntax
 - Unterstützung von Namespaces und mehreren Schemas pro Dokument
 - neue XML-Schemata werden in eigenen Namensraum abgebildet
 - Erweiterbares Typsystem (simple, complex types)
 - Erlaubt Definition einfacher Integritätsbedingungen (unique, key)
 - Komplexer und weniger kompakt gegenüber DTD

XML Schema: vordefinierte Typhierarchie



XML-Schema: simpleType

- **simpleType**: benutzerdefinierter, einfacher Datentyp
 - Kann keine Attribute oder Kind-Elemente enthalten
 - Geeignet zur Definition von XML-Attributwerten und Elementinhalten
- 3 Möglichkeiten zur Bildung einfacher Typen
 - Einschränkung (**xs:restriction**) von Eigenschaften (**xs:facets**)
 - Listenbildung (**xs:list**)
 - Vereinigung (**xs:union**)
- Beispiel: simpleType-Definition mit length-Einschränkung
 - Präfix **xs:** bezeichne standardisierten XML-Schema-Namensraum
- *anonyme vs. benannte Typen*: ohne / mit name-Attribut (ermöglicht Mehrfach-/Wiederverwendung)

```
1 <xs:simpleType name="LoginName">
2   <xs:restriction base="xs:string">
3     <xs:length value="8"/>
4   </xs:restriction>
5 </xs:simpleType>
```

XML-Schema: simpleType (2)

- Möglichkeiten der Einschränkung (**xs:restriction**)
 - Längenbeschränkung für Zeichenketten sowie Listen: **length**, **minLength**, **maxLength**
 - Längenbeschränkung für Dezimal-Beschränkungen: **totalDigits**, **fractionDigits**
 - Wertebereichs-Unter/Obergrenzen: **minInclusive**, **maxInclusive**, **minExclusive**, **maxExclusive**
 - Aufzählung zulässiger Werte: **enumeration**
 - Muster: **xs:pattern value**=„<regulärer Ausdruck>“
- Listenbildung (**xs:list**): definiert leerzeichen-
- Separierte Liste von simpleType-Werten
- Vereinigung (**xs:union**): ermöglicht Typ mit
- Mehrere Wertebereichen

XML-Schema: simpleType (2) - Beispiele

```
1 <xs:simpleType name="Kontakt">
2   <xs:union memberTypes="TelefonTyp EmailTyp"/>
3 </xs:simpleType>
4
5 <xs:simpleType name="BibId">
6   <xs:restriction base="xs:string">
7     <xs:pattern value="[A-Z][1-9][0-9]*"/>
8   </xs:restriction>
9 </xs:simpleType>
10
11 <xs:simpleType name="Lottozahlen">
12   <xs:list>
13     <xs:simpleType>
14       <xs:restriction base="xs:integer">
15         <xs:minInclusive value="1"/>
16         <xs:maxInclusive value="49"/>
17       </xs:restriction>
18     </xs:simpleType>
19   </xs:list>
20 </xs:simpleType>
```

XML-Schema: complexType

- komplexe Datentypen dienen der Beschreibung von Elementinhalten
 - Beschränkung (**restriction**) oder Erweiterung (**extension**) bestehender Typen
 - Einfaches Inhaltsmodell (**simple content**): für Elemente ohne Subelemente, nur Attribute
 - Default: komplexes Inhaltsmodell (**complexContent**), Basistyp: **anyType**
- Elementstrukturierung bei **complexContent**
 - Reihung: **sequence**
 - Auswahl/Alternative: **choice**
 - Gruppe: all (jedes Element kann max. 1-mal auftreten; reihenfolgeunabhängig)
- Kardinalitätsrestriktionen: **minOccurs**, **maxOccurs**
 - Default-Wert: 1
 - Unbeschränkte Anzahl: „unbounded“

XML-Schema: complexType (2)

- Definition durch **Beschränkung** (restriction)

```
1 <xs:complexType name="DozentTyp">
2   <xs:complexContent>
3     <xs:restriction base="xs:anyType">
4       <xs:sequence>
5         <xs:element name="Name" type="xs:string"/>
6         <xs:element name="Einrichtung" type="xs:string"/>
7       </xs:sequence>
8     </xs:restriction>
9   </xs:complexContent>
10 </xs:complexType>
```

```
1 <xs:complexType name="DozentTyp">
2   <xs:sequence>
3     <xs:element name="Name" type="xs:string"/>
4     <xs:element name="Einrichtung" type="xs:string"/>
5   </xs:sequence>
6 </xs:complexType>
```


XML-Schema: complexType (3)

Definition durch **Erweiterung** (extension)

```
1 <xs:complexType name="UniDozentTyp">
2   <xs:complexContent>
3     <xs:extension base="DozentTyp">
4       <xs:sequence>
5         <xs:element name="Adresse" type="AdressTyp"/>
6         <xs:choice>
7           <xs:element name="Mail" type="xs:string"/>
8           <xs:element name="URL" type="xs:string"/>
9         </xs:choice>
10      </xs:sequence>
11    </xs:extension>
12  </xs:complexContent>
13 </xs:complexType>
```

XML-Schema: Schemaaufbau

- Schema in **schema**-Element eingeschlossen
 - Namensraum `http://www.w3.org/2001/XMLSchema` für Schemaelemente
 - Spezifikation eines „targetNamespace“, dem das Schema zugeordnet wird
- Schemakomponenten, die direkt unter schema-Element liegen, sind **global**; tiefer liegende Komponenten sind **lokal**

```
1 <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema" targetNamespace="...">
2   <xs:import>
3   <xs:complexType name="...">...</xs:complexType>
4   <xs:attribute name="..." weitereAttribute="...">
5   <xs:element name="...">
6     <xs:complexType>
7       <xs:sequence>
8         <xs:element name="..." type="datenTyp"/>
9       </xs:sequence>
10      <xs:attribute name="..." type="datenTyp"/>
11    </xs:complexType>
12  </xs:element>
13 </xs:schema>
```

XML-Schema: Elementdeklaration

- verschiedene Möglichkeiten der Elementdeklaration
 - direkte Deklaration des Elementinhalts

```
1 <xs:element name="Dozent">
2   <xs:complexType>
3     <xs:attribute name="Vorname" type="xs:string"/>
4     <xs:attribute name="Nachname" type="xs:string"/>
5     ...
6   </xs:complexType>
7 </xs:element>
```

- Verweis auf komplexen Datentyp

```
1 <xs:element name="Dozent" type="DozentTyp"/>
```

- Verweis auf globale Elementdeklaration

```
1 <xs:element ref="Dozent"/>
```

Vergleich DTD – XML-Schema Elementdeklaration

DTD

```
1 <!ELEMENT Dozent (Titel?, Vorname+, Name, (Vorlesung | Seminar)*)>
```

XML-Schema

```
1 <xs:element name="Dozent">
2   <xs:complexType>
3     <xs:sequence>
4       <xs:element name="Titel" type="xs:string" minOccurs="0"/>
5       <xs:element name="Vorname" type="xs:string" maxOccurs="unbounded"/>
6       <xs:element name="Name" type="xs:string"/>
7       <xs:choice minOccurs="0" maxOccurs="unbounded">
8         <xs:element name="Vorlesung" type="xs:string"/>
9         <xs:element name="Seminar" type="xs:string"/>
10      </xs:choice>
11    </xs:sequence>
12  </xs:complexType>
13 </xs:element>
```

XML-Schema: Attributdeklaration

- Attributdeklarationen
 - global oder lokal am Ende einer complexType-Definition
 - direkte Deklaration oder Referenzierung über *type* bzw. *ref* Typen
 - nur simpleType-Datentypen erlaubt
 - Attribute für Attribute: name, type, use (optional/required), default, fixed

Beispiele

```
1 <xs:element name="Vorlesung">
2   <xs:complexType>
3     <xs:sequence>...</xs:sequence>
4     <xs:attribute name="Uhrzeit" type="xs:time" use="required"/>
5   </xs:complexType>
6 </xs:element>
```

XML-Schema: Attributdeklaration - Beispiele

Beispiel: Aufzählungen

```
1 <xs:attribute name="Farbe" default="blau">
2   <xs:simpleType>
3     <xs:restriction base="xs:NMTOKEN">
4       <xs:enumeration value="rot"/>
5       <xs:enumeration value="gelb"/>
6       <xs:enumeration value="blau"/>
7     </xs:restriction>
8   </xs:simpleType>
9 </xs:attribute>
```

Beispiel: Direkt vs. Referenz

```
1 <xs:attribute name="Einheit" type="xs:string" fixed="km"/>
2
3 <xs:attribute name="id" type="xs:ID" use="required"/>
4
5 <xs:attribute name="Artikel" type="xs:IDREF" use="required"/>
```

XML-Schema: Schlüsselbedingungen

- Eindeutigkeitsbedingung: **unique**
 - Selector-Ausdruck: Teilobjekte eines Dokuments, auf die Bedingung anzuwenden ist (XPath-Ausdruck)
 - field-Ausdruck: Element- bzw. Attributwert, bzgl. dessen (Eindeutigkeit) getestet werden soll

```
1 <xs:element name="VLVerzeichnis">
2   <xs:complexType>...</xs:complexType>
3   <xs:unique name="UniqueBed">
4     <xs:selector xpath="//Vorlesung"/>
5     <xs:field xpath="Thema/text()"/>
6   </xs:unique>
7 </xs:element>
```

XML-Schema: Schlüsselbedingungen (2)

- Schlüsselbedingung: **key**
- Schlüsselreferenz: **keyref**

```
1 <xs:key name="ArtikelKey">
2   <xs:selector xpath="//Produkte/Artikel"/>
3   <xs:field xpath="@aID"/>
4 </xs:key>
5 <xs:keyref name="ArtikelKeyRef" refer="ArtikelKey">
6   <xs:selector xpath="//Verzeichnis/Artikel"/>
7   <xs:field xpath="@artikelID"/>
8 </xs:keyref>
```

```
1 <Produkte>
2   <Artikel aID="A1">
3     <Name> ... </Name>
4   </Artikel>
5   ...
6   <Verzeichnis>
7     <Artikel artikelID="A1">
8       ... </Artikel>
9   </Verzeichnis>
10 </Produkte>
```

- Vergleich zu ID / IDREF
 - anwendbar für Attribute und Elemente
 - anwendbar für viele Typen
 - Einschränkung bzgl. Teildokumenten

XML-Schema und Namensräume

- XML Schema baut auf und unterstützt Namensräume (Namespace=ns)
- Bisherige Schemabeispiele enthielten *unqualifizierte* (default) Definitionen
- Namensraumzuordnung im Schema und im Dokument erforderlich
 - Attribut **targetNamespace** im Schema
 - Verweise im Schema auf definierte Elemente müssen qualifizierte Namen verwenden
 - Definieren mit: **xmlns:prefix**="..." - Aufrufen mit: **prefix:Vorlesung**

```
1 <schema xmlns="http://www.w3.org/2001/XMLSchema"
2   targetNamespace="http://db.th-owl.de/VLV"
3   xmlns:v="http://db.th-owl.de/VLV">
4   <element name="VLVerzeichnis">
5     <complexType>
6       <sequence>
7         <element ref="v:Vorlesung" minOccurs="0"
8           maxOccurs="unbounded"/>
9       </sequence>
10    </complexType>
11  </element>
12  <element name="Vorlesung"> ... </element>
13  <element name="Thema" type="string" /> ...
```

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <VLVerzeichnis xmlns="http://db-th-owl.
3   de/VLV">
4   <Vorlesung Uhrzeit="14:15">
5     <Thema>DB</Thema>
6     <Dozent>
7       <Name>Prof. Bruland</Name>
8       <Einrichtung>TH OWL</Einrichtung>
9     </Dozent>
10  </Vorlesung>
11 </VLVerzeichnis>
```

Nutzung mehrerer Schemata

- Import-Anweisung: Einbindung anderer Schemata / Namensräume
 - Zugriff per Referenz auf globale Komponenten (Elemente, Attribute, Typen) des importierten Schemas
 - Referenz muss qualifizierten Namen verwenden

```
1 <schema xmlns="http://www.w3.org/2001/XMLSchema"
2   targetNamespace="http://db.th-owl.de/VLV"
3   xmlns:v="http://db.th-owl.de/VLV">
4   <element name="VLVerzeichnis">
5     <complexType>
6       <sequence>
7         <element ref="v:Vorlesung"
8           minOccurs="0" maxOccurs="unbounded"/>
9       </sequence>
10    </complexType>
11  </element>
12  <element name="Vorlesung"> ... </element>
13  <element name="Thema" type="string" /> ...
```

```
1 <book xmlns="urn:de.th-owl.bib"
2   xmlns:dc="http://purl.org/dc/elements
3     /1.1/">
4   <dc:title>Web und Datenbanken</dc:
5     title>
6   <dc:publisher>dpunkt -Verlag</dc:
7     publisher>
8   <dc:identifier>3-89864-189-9</dc:
9     identifier>
10  <location>Zweigstelle 1</location>
11  <identifier>ST 6519</identifier>
12 </book>
```

Erweiterungen bestehender Schemata

- Basierend auf einem vorhandenen XML-Schema können folgende Elemente erweitert werden:
 - simpleType, complexType, group, attributeGroup

MySchema1.xsd

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <xs:schema xmlns:xs="http://www.w3.org/2001/
  XMLSchema">
3   <xs:complexType name="pname">
4     <xs:sequence>
5       <xs:element name="firstname"/>
6       <xs:element name="lastname"/>
7     </xs:sequence>
8   </xs:complexType>
9   <xs:element name="customer" type="pname"/>
10 </xs:schema>
```

MySchema2.xsd

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <xs:schema xmlns:xs="http://www.w3.org/2001/
  XMLSchema">
3   <xs:redefine schemaLocation="MySchema1.xsd">
4     <xs:complexType name="pname">
5       <xs:complexContent>
6         <xs:extension base="pname">
7           <xs:sequence>
8             <xs:element name="country"/>
9           </xs:sequence>
10          </xs:extension>
11        </xs:complexContent>
12      </xs:complexType>
13    </xs:redefine>
14    <xs:element name="author" type="pname"/>
15  </xs:schema>
```

Übersicht

1 XML-Technologien

Einführung in XML

XML-Schema (DTD & XSD)

XPath - Zugriff auf XML-Dokumente

```
1 //TH-OWL/FB5/Dozenten[@name="Bruland"]/Vorlesung[@kurz="DB"]/@Uhrzeit
```

XPath

- „Navi durch eine XML-Struktur“
- Adressierung von Teilen eines Dokumentes
- Formulierung von Bedingungen an diese Knotenmengen
- XPath-Ausdruck kann aus mehreren Schritten bestehen, Kopplung über /
- Ergebnis eines Schrittes sind Knotenmengen (auch die leere Knotenmenge ist möglich) oder einzelne Werte
- Abarbeitung erfolgt von links beginnend

XPath - Elemente

XML-Dokument wird als Baum mit folgenden Knotenarten betrachtet:

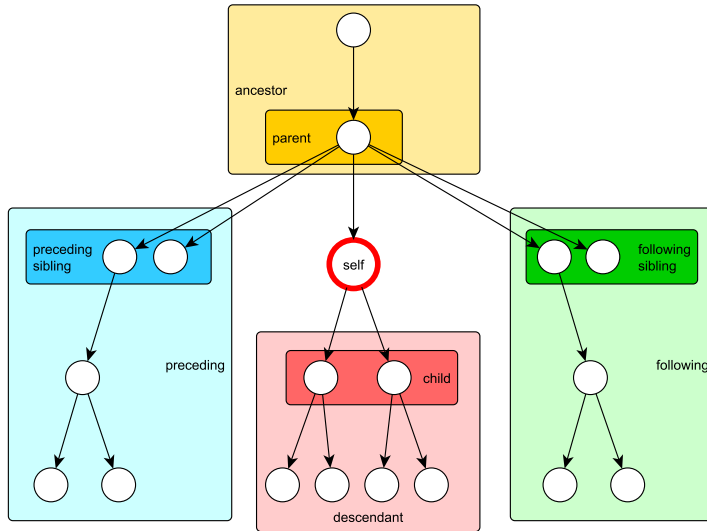
- Prozessinstruktion-Knoten
- Namensraumknoten
- Wurzelknoten
- Elementknoten
- Attributknoten
- Textknoten
- Kommentarknoten

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <Kreditkarte xmlns="http://www.creditcard.org"
3   Herausgeber="Mastercard" Nummer="1234-5678-9012-3456"
4     Waehrung="EURO">
5   <Inhaber Name="Mustermann" Vorname="Max" maennlich="true"
6     Partner="null">
7     <Hobbys>
8       <Hobby>Reiten</Hobby>
9       <Hobby>Golfen</Hobby>
10      <Hobby>Lesen</Hobby>
11    </Hobbys>
12    <Kinder/>
13  </Inhaber>
14 </Kreditkarte>
```

Aufbau eines Navigationsschrittes

- Absolute und relative Pfade möglich
 - Absolute Pfade beginnen von der Wurzel des XML-Dokumentes
 - Relative gehen vom Kontextknoten (=aktueller Knoten) aus
- Ein Schritt besteht aus: `achse::knotentest[prädikat]`
 - Achse:
 - Beziehung zwischen dem Kontextknoten und dem auszuwählenden Knoten
 - Knotentest:
 - Legt den Knotentyp und den Namen der zu selektierenden Knoten fest
 - Zum Beispiel: Knotentyp: Element oder Attribut, Name jeweils wie angegeben, * für alle
 - Prädikat
 - Ein, kein oder mehrere Prädikate schränken die Knotenmenge ein
 - (na ja: kein Prädikat schränkt die Knotenmenge natürlich nicht ein)
- Beispiele:
 - `child::href[position()=2]`
 - `child::author[last()]`
 - `attribute::autoname`
 - `parent::*`

XPath – Zugriffsachsen in XML-Baumstruktur



XPath – Zugriffsachsen in XML-Baumstruktur

Vorwärts

- child
- descendant
- attribute
- self
- descendant-or-self
- following-sibling
- following
- namespace

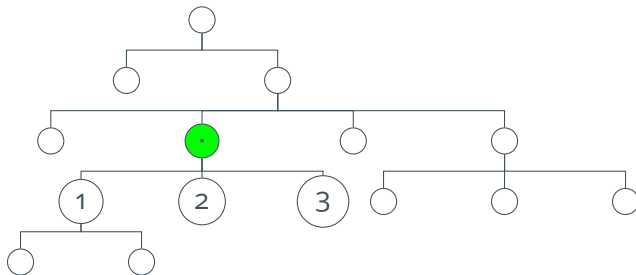
Rückwärts

- parent
- ancestor
- preceding-sibling
- preceding
- ancestor-or-self

child - Kind-Ebene

child enthält die Kinder des Kontextknotens in der Reihenfolge der Dokumentordnung.

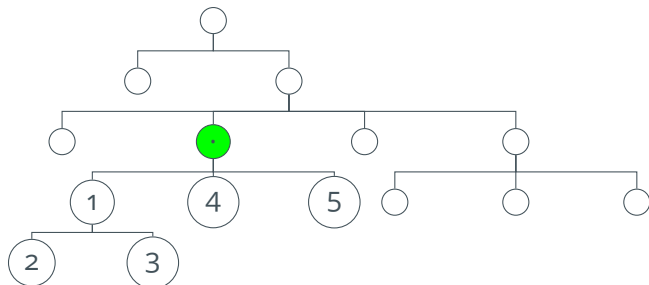
```
1 <?xml version="1.0"?>
2 <Arztpraxis>
3   <Arzt Name="Dr. G. Huber">
4     <Patient PID="478596">
5       <VName>Petra</VName>
6       <Name>Poolmann</Name>
7       <Sex>female</Sex>
8       <GebDat>1987-08-12</GebDat>
9       <Krankengeschichte>
10        <KH ICD="S00.31">Schürfwunde</KH>
11        <KH ICD="I21.3">Herzinfarkt</KH>
12      </Krankengeschichte>
13    </Patient>
14    ...
```



descendant - Kindeskind-Ebene(n)

descendant enthält sämtliche Nachkommen des Kontextknotens in der Reihenfolge der Dokumentordnung. Dazu zählen Kinder und Kindeskind, aber keine Attribut- oder Namensraumknoten.

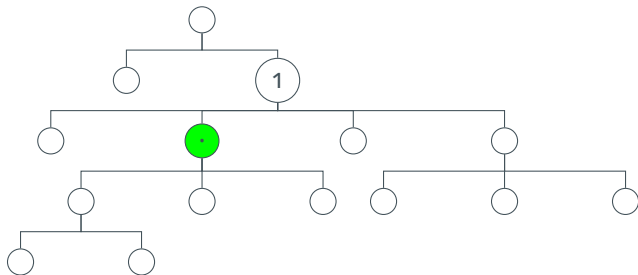
```
1 <?xml version="1.0"?>
2 <Arztpraxis>
3   <Arzt Name="Dr. G. Huber">
4     <Patient PID="478596">
5       <VName>Petra</VName>
6       <Name>Poolmann</Name>
7       <Sex>female</Sex>
8       <GebDat>1987-08-12</GebDat>
9       <Krankengeschichte>
10        <KH ICD="S00.31">Schürfwunde</KH>
11        <KH ICD="I21.3">Herzinfarkt</KH>
12      </Krankengeschichte>
13    </Patient>
14    ...
```



parent - Eltern-Ebene

parent enthält den Elternknoten des Kontextknotens, also den direkt übergeordneten Knoten.

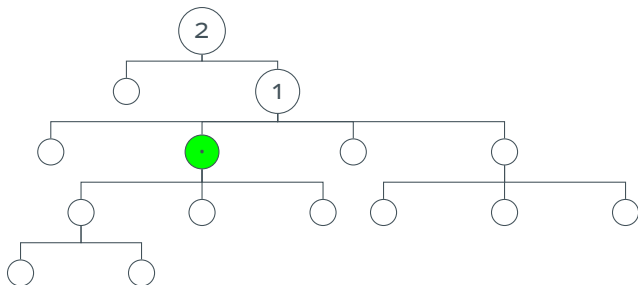
```
1 <?xml version="1.0"?>
2 <Arztpraxis>
3   <Arzt Name="Dr. G. Huber">
4     <Patient PID="478596">
5       <VName>Petra</VName>
6       <Name>Poolmann</Name>
7       <Sex>female</Sex>
8       <GebDat>1987-08-12</GebDat>
9       <Krankengeschichte>
10        <KH ICD="S00.31">Schürfwunde</KH>
11        <KH ICD="I21.3">Herzinfarkt</KH>
12      </Krankengeschichte>
13    </Patient>
14    ...
```



ancestor - Elterneltern-Ebene(n)

ancestor enthält sämtliche Vorfahren des Kontextknotens in umgekehrter Dokumentordnung. Vorfahren sind Elternknoten sowie die Eltern der Elternknoten und immer der Wurzelknoten.

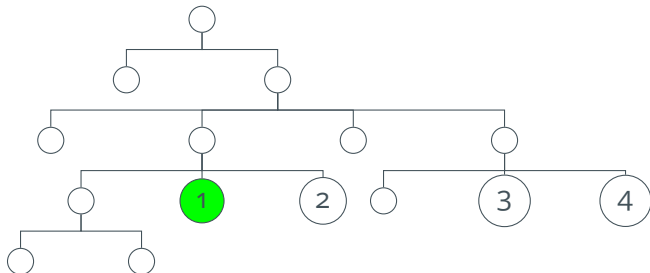
```
1 <?xml version="1.0"?>
2 <Arztpraxis>
3   <Arzt Name="Dr. G. Huber">
4     <Patient PID="478596">
5       <VName>Petra</VName>
6       <Name>Poolmann</Name>
7       <Sex>female</Sex>
8       <GebDat>1987-08-12</GebDat>
9       <Krankengeschichte>
10        <KH ICD="S00.31">Schürfwunde</KH>
11        <KH ICD="I21.3">Herzinfarkt</KH>
12      </Krankengeschichte>
13    </Patient>
14    ...
```



following-sibling - Folgende-Geschwister-Ebene

following-sibling enthält sämtliche nachfolgenden Geschwister des Kontextknotens auf selber Ebene. Für Attribut- oder Namensraumknoten ist diese Achse leer.

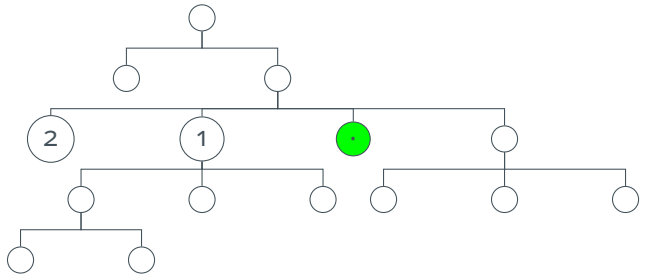
```
1 <?xml version="1.0"?>
2 <Arztpraxis>
3   <Arzt Name="Dr. G. Huber">
4     <Patient PID="478596">
5       <VName>Petra</VName>
6       <GebDat>1987-08-12</GebDat>
7       <Krankengeschichte>
8         <KH ICD="S00.31">Schürfwunde</KH>
9         <KH ICD="I21.3">Herzinfarkt</KH>
10      </Krankengeschichte>
11    </Patient>
12    <Patient PID="478596">
13      <VName>Petra</VName>
14      <GebDat>1987-08-12</GebDat>
15      <Krankengeschichte>
16        <KH ICD="S00.31">Schürfwunde</KH>
17        <KH ICD="I21.3">Herzinfarkt</KH>
18      </Krankengeschichte>
19    </Patient>
20    ...
```



preceding-sibling - Vorherige-Geschwister-Ebene

preceding-sibling enthält sämtliche vorhergehenden Geschwister des Kontextknotens. Für Attribut-/Namensraumknoten ist diese Achse leer.

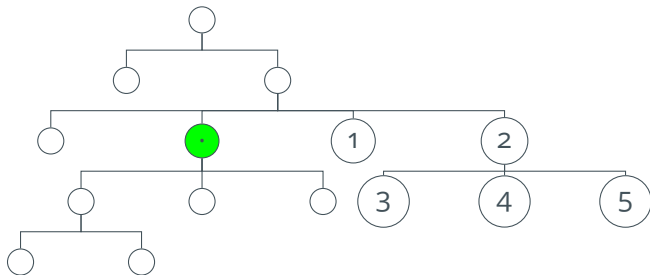
```
1 <?xml version="1.0"?>
2 <Arztpraxis>
3   <Arzt Name="Dr. G. Huber">
4     <Patient PID="478596">
5       <VName>Petra</VName>
6       <Name>Poolmann</Name>
7       <Sex>female</Sex>
8       <GebDat>1987-08-12</GebDat>
9       <Krankengeschichte>
10        <KH ICD="S00.31">Schürfwunde</KH>
11        <KH ICD="I21.3">Herzinfarkt</KH>
12      </Krankengeschichte>
13    </Patient>
14    ...
```



following - Gesamte Nachfolger-Ebene(n)

following enthält alle nachfolgenden Knoten des Kontextknotens ohne seine Nachkommen und ohne Attribut- und Namensraumknoten.

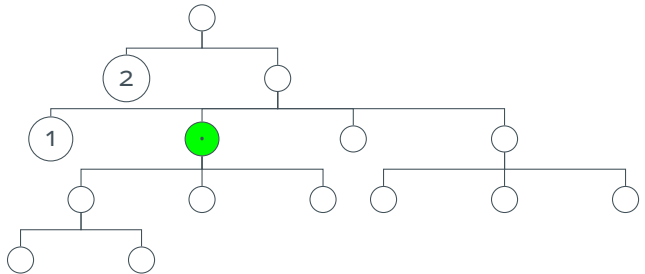
```
1 <?xml version="1.0"?>
2 <Arztpraxis>
3   <Arzt Name="Dr. G. Huber">
4     <Patient PID="478596">
5       <VName>Petra</VName>
6       <Name>Poolmann</Name>
7       <Sex>female</Sex>
8       <GebDat>1987-08-12</GebDat>
9       <Krankengeschichte>
10        <KH ICD="S00.31">Schürfwunde</KH>
11        <KH ICD="I21.3">Herzinfarkt</KH>
12      </Krankengeschichte>
13    </Patient>
14    ...
```



preceding - Gesamte Vorgänger-Ebene(n)

preceding enthält alle vorangehenden Knoten des Kontextknotens ohne seine Vorfahren und ohne Attribut- und Namensraumknoten.

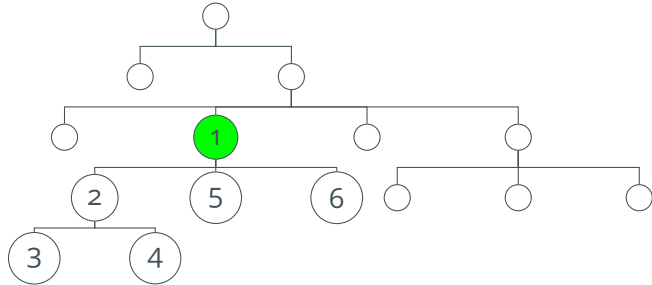
```
1 <?xml version="1.0"?>
2 <Arztpraxis>
3   <Arzt Name="Dr. G. Huber">
4     <Patient PID="478596">
5       <VName>Petra</VName>
6       <Name>Poolmann</Name>
7       <Sex>female</Sex>
8       <GebDat>1987-08-12</GebDat>
9       <Krankengeschichte>
10        <KH ICD="S00.31">Schürfwunde</KH>
11        <KH ICD="I21.3">Herzinfarkt</KH>
12      </Krankengeschichte>
13    </Patient>
14    ...
```



descendant-or-self - Konten und sämtliche Nachkommen

descendant-or-self enthält den Kontextknoten und sämtliche Nachkommen.

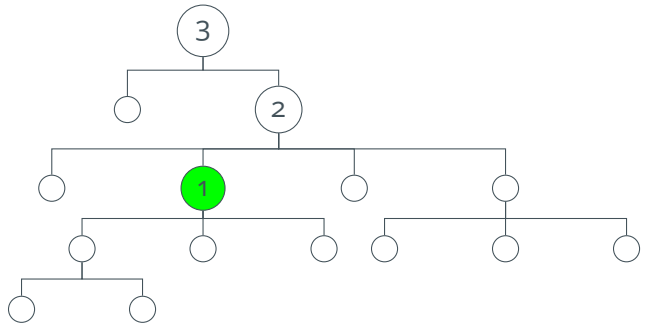
```
1 <?xml version="1.0"?>
2 <Arztpraxis>
3   <Arzt Name="Dr. G. Huber">
4     <Patient PID="478596">
5       <VName>Petra</VName>
6       <Name>Poolmann</Name>
7       <Sex>female</Sex>
8       <GebDat>1987-08-12</GebDat>
9       <Krankengeschichte>
10        <KH ICD="S00.31">Schürfwunde</KH>
11        <KH ICD="I21.3">Herzinfarkt</KH>
12      </Krankengeschichte>
13    </Patient>
14    ...
```



ancestor-or-self - Knoten und sämtliche Vorfahren

ancestor-or-self enthält den Kontextknoten und sämtliche Vorfahren.

```
1 <?xml version="1.0"?>
2 <Arztpraxis>
3   <Arzt Name="Dr. G. Huber">
4     <Patient PID="478596">
5       <VName>Petra</VName>
6       <Name>Poolmann</Name>
7       <Sex>female</Sex>
8       <GebDat>1987-08-12</GebDat>
9       <Krankengeschichte>
10        <KH ICD="S00.31">Schürfwunde</KH>
11        <KH ICD="I21.3">Herzinfarkt</KH>
12      </Krankengeschichte>
13    </Patient>
14    ...
```



Achsentypen

- Die Achse **attribute** enthält die Attribute des Kontextknotens, wobei der Kontextknoten in jedem Fall ein Elementknoten sein muss.
- Die Achse **namespace** enthält alle Namensraumknoten des Kontextknotens, wobei der Kontextknoten in jedem Fall ein Elementknoten sein muss.
- Die Achse **self** enthält nur den Kontextknoten.

Achsentypen - Kurzformen

Kurzform	Langform	Bedeutung
	child::	Child ist Standardachse
.	self::node()	aktueller Knoten
..	parent::node()	Vaterknoten
//	/descendant-or-self::node()	Nachfahren des Kontextknotens und dieser selbst, Sprich: Beliebiger Knoten im Dokument, ohne vorerst die gesamte Hierarchie aufzulisten.
@	attribute::	Attribute des Kontextknotens
*		Knoten mit beliebigem Namen (Wildcard)
@*		Bezeichnet alle Attribute des Kontextknotens

Prädikate

- Prädikat wird für jeden Knoten ausgewertet
- Knoten bei denen das Ergebnis true ist, werden in das Ergebnis übernommen
 - Vorhandensein von Werten (Werteselektion)

```
1 /buch[@erscheinungsjahr=2000]  
2 /hotel/adresse/ort[text()='Rostock']
```

- Strukturselektion – basiert auf strukturellen Eigenschaften (im Gegensatz zu Datenbanken, dort nur Werteselektion)

```
1 /hotel[kategorie], hotel[//ort] <!-- Vorhandensein von Elementen -->  
2 /hotel[@url] <!-- Vorkommen von Attributen -->
```

- Positionsangabe: Wie im Array (1-basiert) oder über Funktion (first(), last())

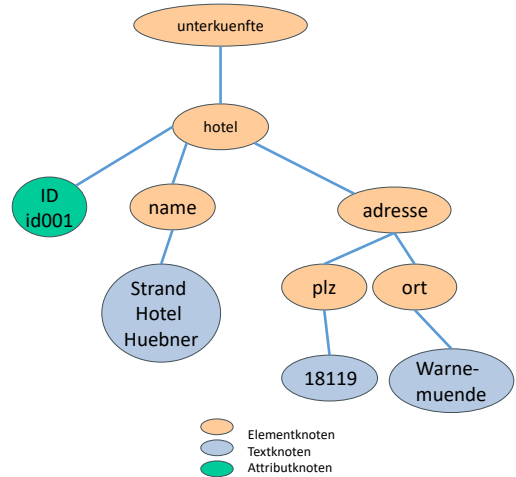
```
1 /hotel[3], ort[1], autor[last()]
```

Prädikate (2)

- Reihenfolge der Prädikatauswertung ist relevant!
- Verbindungen von Prädikaten ist durch
 - and, or (Ergebnis boolescher Wert)
 - =, >, >=, <, <= (Ergebnis boolescher Wert)
 - +, -, *, mod, div (Ergebnis numerischer Wert)
 - | dient zur Vereinigung von Knotenmengen
 - () Klammerung beeinflusst die Ausführungsreihenfolge
- XPath-Funktionen (einige waren schon in Beispielen zu sehen)
 - last()
 - position() – auch als Kurzform [n]
 - contains(string, string)

XPath - Beispiel

```
1 <unterkuenfte>
2 <hotel id="id001">
3   <name>Strand Hotel Huebner</name>
4   <adresse>
5     <plz>18119</plz>
6     <ort>Warnemuende</ort>
7     ...
8   </adresse>
9 </hotel>
10 ..
11 </unterkuenfte>
12
13 <!-- Abfragen -->
14 /unterkuenfte/hotel[adresse/ort/text()="Warnemuende"
15   or adresse/ort/text()="Rostock"]/name/text()
16 /unterkuenfte/hotel[contains(name/text(),"Strand")]
```



Knoten am Beispiel

Funktion	Beschreibung	Beispiel	Ergebnis
Name	Wählt einen Knoten anhand seines Namens aus.	//child::Name	<Name>
Asteriskus *	Wählt alle Knoten-Typen entlang der Achse aus.	//Zeitraum/ancestor::*	<Tarif>
text()	Wählt alle Textinhalte der Knoten aus.	//Name/text()	Mondschein2
comment()	Wählt Kommentarinhalte in der Achse aus.	/comment()	Erstellt: 14.01.2014
processing-instruction()	Wählt die Inhalte der Processing-Anweisungen aus	//processing-instruction() //processing-instruction("xml-stylesheet")	type="text/css" href="stile/tarife.css"

```

1  <?xml version="1.0" encoding="ISO-8859-1"?>
2  <!-- Erstellt: 15.01.2004 -->
3  <Tarif xmlns="rf:www.ruhrfon.biz">
4    <?xml-stylesheet type="text/css" href="stile/
      tarife.css"?>
5      <Name Typ="p">Mondschein2</Name>
6      <Zeitraum>
7        <GueVon>01.01.04</GueVon>
8        <GueBis>31.12.04</GueBis>
9      </Zeitraum>
10     <Uhrzeit>
11       <GueVon>20</GueVon>
12       <GueBis>6</GueBis>
13     </Uhrzeit>
14   </Tarif>

```

XPath - Knotenmengenfunktionen

Funktion	Beschreibung	Beispiel
<code>position()</code>	Ermittelt die aktuelle Position	<code>//Element[position()=4]</code> liefert das 4. Element
<code>last()</code>	Liefert den letzten Knoten eines Knotensets	<code>//Element[last()]</code> liefert den Knoten des letzten Elements
<code>count(Knotenmenge)</code>	Anzahl an Knoten in ausgewählter Knotenmenge	<code>count(Element)</code> liefert die Anzahl an Element zurück
<code>name(Knotenmenge)</code>	Wählt Knoten mit bestimmtem Namen (und Namespace) aus	<code>//*[name()="Name"]</code>

XPath - Knotenmengenfunktionen

Funktion	Beschreibung	Beispiel
<code>local-name(Knotenmenge)</code>	Wählt Knoten wie <code>name()</code> allerdings ohne Namespace.	<code>local-name(namespace:bspknoten)</code> liefert den Namen: <code>bspknoten</code>
<code>namespace-uri(Knotenmenge)</code>	Liefert den URI-Wert für einen Namespace	<code>namespace-uri(\$in-xml)</code> → zero-length xs:anyURI value the no- Namespace element itself <code>namespace-uri(\$in-xml//pre:prefixed)</code> → <code>http://datypic.com/pre</code>
<code>id()</code>	Ermittelt die Knotenmengen mit dem angegebenen ID-Wert	<code>\$book/id('preface')</code> → <section id="preface">This book introduces XQuery... The examples are downloadable<fnref ref="fn1"/>... </section>

XPath - String-Funktionen

Funktion	Beschreibung	Beispiel
<code>starts-with(zk1, zk2)</code>	prüft, ob ein Item (das als String gecastet werden kann) mit einer bestimmten Zeichenfolge beginnt.	<code>starts-with('query', 'que') → true</code> <code>starts-with('query', 'u') → false</code>
<code>contains(zk1, zk2)</code>	Erweitert vorherige Funktion und liefert auch dann einen Rückgabewert true, wenn mehrere Buchstaben angegeben werden und diese an einer beliebigen Stelle verborgen sind.	<code>contains('query', 'ery') → true</code> <code>contains('query', 'x') → false</code>
<code>substring-before(zk1, zk2)</code>	Ermittelt die Zeichen einer Zeichenkette 1 vor der Position, die in Zeichenkette 2 angegeben wurde.	<code>substring-before('Autobahn', 'b') → Auto</code> <code>substring-before('Autobahn', 'x') → "</code>

XPath - String-Funktionen

Funktion	Beschreibung	Beispiel
<code>substring-after(zk1, zk2)</code>	Ermittelt die Zeichen einer Zeichenkette 1 vor der Position, die in Zeichenkette 2 angegeben wurde.	<code>substring-after('Autobahn', 'b') → ahn</code> <code>substring-after('Autobahn', 'x') → "</code>
<code>substring(zk1, zk2, length)</code>	Ermittelt die Zeichen einer Zeichenkette 1 vor der Position, die in Zeichenkette 2 angegeben wurde, für die Länge <i>length</i> .	<code>substring('query', 3) → ery</code> <code>substring('query', 2, 3) → uer</code> <code>substring('query', -2, 5) → qu</code>
<code>string-length(xs:string?)</code>	Ermittelt von einer Zeichenkette die Länge.	<code>string-length('query') → 5</code> <code>string-length(' query ') → 9</code>
<code>string(Objekt)</code>	Transformiert ein Objekt in eine Zeichenkette.	<code>string(\$in-xml-2/number) → 784</code>
<code>concat(zk1, [zk2,]*)</code>	Verknüpft die eingehenden Zeichenketten.	<code>concat('a', 'b', 'c') → abc</code> <code>concat('a', (), 'b', ", 'c') → abc</code>

XPath – Logische Funktionen

Funktion	Beschreibung	Beispiel
<code>true()</code>	Liefert immer den Wert wahr.	<code>true() → true</code>
<code>false()</code>	Liefert immer den Wert falsch.	<code>false() → false</code>
<code>not(Objekt)</code>	Liefert den Wert wahr, wenn das Objekt falsch ist.	<code>not(32 > 20) → false</code> <code>not(true()) → false</code>
<code>boolean(Objekt)</code>	Transformiert ein Objekt in einen logischen Wert, wobei nicht leere Knotenmengen und Zeichenketten sowie Zahlen größer 0 den Wert wahr ergeben.	<code>boolean(0 1) → false true</code>
<code>lang(zk)</code>	Liefert den Wert wahr, wenn der <code>xml:lang</code> -Wert mit der in der Zeichenkette angegebenen Sprache übereinstimmt.	<code>\$in-xml//desc[lang('FR')] →</code> <code><desc xml:lang="fr"></code> <code> <line>Une ligne de texte.</line></code> <code></desc></code>

XPath - Numerische Funktionen

Funktion	Beschreibung	Beispiel
<code>number(Objekt)</code>	Transformiert ein Wert oder einen Knoten in eine Zahl.	<code>number('29.99') → 29.99</code> <code>number('ABC') → NaN (Not a Number)</code>
<code>sum(Knotenmenge)</code>	Berechnet die Summe der zu Zahlen transformierten Werte der Knotenmenge.	<code>sum((1, 2, 3, ())) → 6</code> <code>sum(//item/@quantity) → 7</code>
<code>floor(numeric?)</code>	Liefert die vorher liegende Ganzzahl.	<code>floor(5.1) → 5</code> <code>floor(5.7) → 5</code> <code>floor(-5.1) → -6</code>
<code>ceiling(numeric?)</code>	Liefert die nächstgrößere Ganzzahl.	<code>ceiling(5.1) → 6</code> <code>ceiling(5.5) → 6</code> <code>ceiling(-5.5) → -5</code>
<code>round(numeric?)</code>	Liefert den gerundeten Wert zur nächsten Ganzzahl.	<code>round(5.1) → 5</code> <code>round(5.5) → 6</code> <code>round(-5.5) → -5</code>

XPath - Zeichenkettenfunktionen

Funktion	Beschreibung	Beispiel
<code>string-to-codepoints(xs:string?)</code>	Gibt die Unicode-Integer-Werte der einzelnen Zeichen zurück.	„Hallo“ → (72, 97, 108, 108, 111)
<code>codepoints-to-string((xs:integer*))</code>	Gibt die Zeichen zu den jeweiligen Unicode-Werten zurück. (Hier mit doppelter runder Klammer)	„(83, 105, 109, 111, 110, 101)“ → S i m o n e
<code>compare(vgl1 xs:string?, vgl2 xs:string?, Vergleichs- modifikator xs:string)</code>	Vergleicht 2 Werte auf Basis ihrer Unicodes mit folgenden Ergebnissen: -1 vgl1 < vgl2 0 vgl1 = vgl2 1 vgl1 > vgl2	<code>compare('lassdas', 'laßdas', 'http://www.w3.org/2013/collation/ UCA?lang=de;strength=primary')</code> → 0, da beide Zeichenketten gleich sind

XPath - Aggregatfunktionen

- `fn:sum((1,2,3))` → 6
- `fn:avg((1,2,3))` → 2
- `fn:max((1,2,3))` → 3
- `fn:min((1,2,3))` → 1

Zusammenfassung XPath

- Navigation durch den Baum, der das XML-Dokument darstellt (über Navigationsachsen)
- schrittweise Abarbeitung
- Ergebnis jedes Schrittes sind Knotenmengen
- auf diesen können durch Prädikate Bedingungen formuliert werden
-
- Finden Anwendung bei
 - XQuery
 - XSLT
 - ...
- Detaillierte Informationen zu Funktionen und Aufbau finden Sie hier:
 - <https://www.w3.org/TR/xpath-functions-31/>

Weiterführende Literatur

- Wilfried Grupe. XML: Grundlagen, Technologien, Validierung, Auswertung. MITP, 2018. ISBN: 978-3-958-45754-6 [https://th-owl.digibib.net/search/katalog/record/\(DE-605\)HT019681078](https://th-owl.digibib.net/search/katalog/record/(DE-605)HT019681078)
- Helmut Vonhoegen. Einstieg in XML: Grundlagen, Praxis, Referenzen, 2013 [https://th-owl.digibib.net/search/katalog/record/\(DE-605\)HT017686710](https://th-owl.digibib.net/search/katalog/record/(DE-605)HT017686710)
- https://www.w3schools.com/xml/xsl_intro.asp
- <http://www.xsltfunctions.com/>
- M Klettke, H Meyer. XML Datenbanken – Konzepte, Sprachen und Systeme. Dpunkt.verlag. 2003, ISBN 3-89864-148-1 <https://th-owl.digibib.net/search/eds/record/edsbas:edsbas.D9F42D5D>

Vielen Dank!